Reimplementing TransVG and Improving Visual Grounding with Decoder-Inspired Sequential Coordinate Prediction

Le Minh Khoi[1,2], Bui Quoc Bao[1,2], Huynh Phat Dat[1,2],
[1]*Faculty of Information Science and Engineering, University of Information Technology, Ho Chi Minh City, Vietnam*
[2]*Vietnam National University, Ho Chi Minh City, Vietnam*
{23520767, 23520091, 24520270}@gm.uit.edu.vn,

***Abstract*—Visual grounding aims to localize the image region referred by a natural language expression. In this project, we first re-implement TransVG, an end-to-end transformer-based visual grounding framework that directly regresses bounding box coordinates from visual and linguistic inputs. During the period of re implementation, we further investigated a Decoder-inspired improvement that reformulates bounding box prediction as a sequence generation problem over discrete coordinate tokens. The proposed direction replaces the regression head with a lightweight transformer decoder and optimizes the model using cross-entropy loss. Due to hardware limitations, all experiments are conducted only on the RefCOCO dataset with a reduced training schedule. The experimental results show that the reimplemented TransVG baseline achieves 46.58%, 51.30%, and 39.23% Precision@0.5 on the validation, testA, and testB splits, respectively. In comparison, the proposed method obtains 74.65%, 79.32%, and 67.67% on the same splits, demonstrating a significant improvement over the reproduced baseline. Although the proposed method has not yet reached the performance of the original TransVG trained for 90 epochs, the results indicate that sequential coordinate prediction is a promising direction for improving visual grounding under limited training resources. Qualitative examples further show that the proposed method can produce more accurate bounding boxes and better align language expressions with visual regions.**

***Index Terms*—Visual grounding, referring expression comprehension, TransVG, transformer, coordinate regression, sequence prediction.**

I. Introduction

VISUAL grounding is a vision-language task in which a model receives an image and a natural language query, and predicts the bounding box of the object or region described by the query. This task is important for building systems that can connect human language with visual perception, such as human-computer interaction, robotic instruction following, and image retrieval.

Traditional visual grounding methods are commonly categorized into two-stage and one-stage pipelines as shown in Fig. 1. Two-stage methods first generate region proposals and then exploit region-expression matching to find the best one, while one-stage methods perform visual-linguistic fusion at intermediate layers of an object detector and output the box with the maximal score over pre-defined dense anchors. Transformer-based approaches simplify this pipeline by allowing visual and textual tokens to interact through self-attention.

Despite the effectiveness, these complicated fusion modules are built on certain pre-defined structures of language queries or image scenes, inspired by the human prior. Typically, the involvement of manually-designed mechanisms in fusion module makes the models overfit to specific scenarios, such as certain query lengths and query relationships, and limits the plenitudinous interaction between visual linguistic contexts. Since these methods' predictions are made out of candidates, the performance is easily influenced by the prior knowledge to generate proposals (or pre-defined anchors) and by the heuristics to assign targets to candidates.

This project focuses on two representative transformer-based visual grounding methods. First, TransVG is reimplemented as the baseline method. The Authors of TransVG show that directly regressing the box coordinates works better than previous methods to ground the queried object indirectly. TransVG formulates visual grounding as a direct coordinate regression problem and uses a visual-linguistic transformer to model cross-modal relations. Second, instead of regressing continuous coordinates directly, our method represents grounding information as a sequence of discrete coordinate tokens and predicts them autoregressively.

The contributions of this report can be summarized as follows:

- We provide a detailed study of TransVG, including data preprocessing, model architecture, training objective, and evaluation protocol.
- We design an improved visual grounding framework by integrating sequential coordinate prediction inspired by Transformer Decoder.
- We conduct experiments to compare the reimplemented baseline and the proposed improvement under the same dataset and training setting.

II. Related Work

A. Two-Stage Methods

Two-stage methods solve visual grounding in two separate steps. In the first step, the model generates a set of candidate regions, also called region proposals, from the input image. These proposals are possible object locations that may contain the target object. In the second step, the model compares

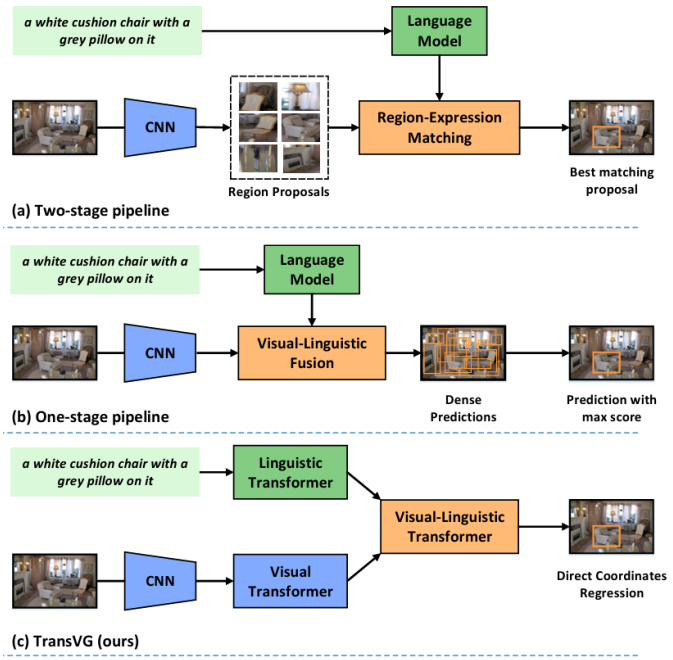


Fig. 1. A comparison of (a) two-stage pipeline, (b) one-stage pipeline, and (c) TransVG framework. TransVG per forms intra-modality and inter-modality relation reasoning with a stack of transformer layers in a homogeneous way, and grounds the object by directly regressing the box coordinates.

each candidate region with the language query and selects the region that best matches the expression. Therefore, two-stage methods treat visual grounding as a region-selection problem: the model first finds possible objects, and then uses the text to choose the correct one. This design can be effective, but it depends heavily on the quality of the generated proposals. If the correct object is not included in the proposal set, the model cannot select it in the second stage. In addition, generating and processing many proposals usually makes two-stage methods more computationally expensive than one-stage methods.

B. One-Stage Methods

Instead of first producing many candidate regions and then matching them with the language query, one-stage methods remove the computation-intensive proposal generation in two-stage methods. one-stage methods combine the language information directly with the visual features extracted from the image. After this fusion, the model predicts the bounding box directly from the language-aware visual feature maps. In other words, the model uses the text query to guide the detection process from the beginning, so it can locate the referred object in a single pipeline. Compared with two stage methods, one-stage methods are usually faster because they do not need to generate and process region proposals. However, they still often rely on dense anchors or sliding-window predictions, so their performance can still be affected by predefined detection structures.and directly perform visual-linguistic fusion on feature maps. These methods are faster, but they often require carefully designed fusion modules and anchor-based prediction strategies.

C. Transformer

Transformer is first proposed in to tackle the neural machine translation (NMT). The primary component of a transformer layer is the attention module, which scans through the input sequence in parallel and aggregates the information of the whole sequence with adaptive weights.Compared to the recurrent units in RNNs, the attention mechanism exhibits better performance in processing long sequences. This superiority has attracted a surge of research interest in applications of transformers in NLP tasks and speech recognition.

Transformer in Vision Tasks. After achieving strong results in language tasks, Transformers have also been applied to computer vision. For example, DETR formulates object detection as a set prediction problem and uses learnable object queries together with attention to model global context and object relationships. ViT further shows that a pure Transformer architecture can perform well for image classification. Other Transformer-based models have also been developed for low-level vision tasks such as image denoising, super-resolution, and deraining.

Transformer in Vision-Language Tasks. Transformers have also been explored for vision-language tasks, especially after the success of BERT. Many visual-linguistic pre-training methods aim to learn joint representations of images and text. These models usually take image regions and text tokens as input, then use Transformer encoder layers to learn the relationship between them. They are often trained with tasks such as image-text matching, word-region alignment, masked language modeling, and masked region modeling.

III. Problem Formulation

Given an input image I and a natural language expression $Q = \{w_1, w_2, ..., w_T\}$, the goal of visual grounding is to predict a bounding box b that localizes the region referred to by the expression. The bounding box can be represented as

$$b = (x_1, y_1, x_2, y_2), \tag{1}$$

where (x_1, y_1) and (x_2, y_2) denote the top-left and bottom-right coordinates, respectively.

The standard evaluation metric is Precision@0.5, where a prediction is considered correct if the intersection-over-union (IoU) between the predicted box and the ground-truth box is greater than 0.5:

$$IoU(b, \hat{b}) = \frac{|b \cap \hat{b}|}{|b \cup \hat{b}|}. \tag{2}$$

IV. Baseline Method: TransVG Reimplementation

A. The Idea

Given an image and a language expression as input, the authors first separate them into two sibling branches, a visual branch and a linguistic branch, to generate visual and linguistic feature embedding. Then, they put the embedding multimodal feature together and append a learnable token (named [REG] token) to construct the inputs of visual-linguistic fusion modules. The visual-linguistic transformer homogeneously embeds the input tokens from different modalities into a common

semantic space by mod eling intra-modality and inter-modality context with the self attention mechanism. Finally, the output state of the [REG] token is leveraged to directly predict the 4-dim coordinates of a referred object in the prediction head.

B. Preliminary

In this section, we will briefly review necessary knowledges which will help readers to easily access to TransVG architecture.

Transformer Encoder. Before describing the architecture of TransVG, we first review the basic transformer encoder. The core component of a transformer is the attention mechanism. Given the query embedding f^q, key embedding f^k, and value embedding f^v, the output of a single-head attention layer is computed as:

$$\mathrm{Attn}(f^q, f^k, f^v) = \mathrm{softmax}\left(\frac{f^q f^k}{\sqrt{d^k}}\right) \cdot f^v, \tag{3}$$

where d^k denotes the channel dimension of the key embedding f^k.

The original transformer follows an encoder-decoder architecture. However, TransVG only adopts transformer encoder layers. Each encoder layer consists of two main sub-layers: a multi-head self-attention layer and a feed-forward network (FFN). In self-attention, the query, key, and value are generated from the same input embedding, allowing each token to interact with all other tokens in the sequence. The FFN is implemented as a multi-layer perceptron composed of fully connected layers and non-linear activation functions.

In each transformer encoder layer, both the self-attention module and the FFN are wrapped with residual connections and layer normalization. Let x_n be the input of the n-th encoder layer. The computation process can be written as:

$$x'_n = \mathrm{LN}(x_n + \mathcal{F}_{\mathrm{MSA}}(x_n)), \tag{4}$$

$$x_{n+1} = \mathrm{LN}(x'_n + \mathcal{F}_{\mathrm{FFN}}(x'_n)), \tag{5}$$

where $\mathrm{LN}(\cdot)$ denotes layer normalization, $\mathcal{F}_{\mathrm{MSA}}(\cdot)$ represents the multi-head self-attention layer, and $\mathcal{F}_{\mathrm{FFN}}(\cdot)$ represents the feed-forward network.

Smooth L1 Loss. Smooth L1 loss is a regression loss function commonly used in bounding box prediction. It is closely related to the Huber loss, which was introduced by Peter J. Huber in 1964 for robust estimation. In the context of object detection, Smooth L1 loss was widely adopted for bounding box regression, especially after its use in Fast R-CNN. The main idea of this loss is to combine the advantages of both L_1 loss and L_2 loss. For small prediction errors, it behaves like L_2 loss, which provides smooth gradients and helps stable optimization. For large prediction errors, it behaves like L_1 loss, which makes it less sensitive to outliers than pure L_2 loss.

Let $b = (x, y, w, h)$ be the predicted bounding box and $\hat{b} = (\hat{x}, \hat{y}, \hat{w}, \hat{h})$ be the ground-truth bounding box. The Smooth L1 loss between b and $\hat{b}$ can be defined as:

$$\mathcal{L}_{smooth\text{-}L1}(b, \hat{b}) = \sum_{i \in \{x,y,w,h\}} smooth_{L1}(b_i - \hat{b}_i), \tag{6}$$

where $b_i - \hat{b}_i$ represents the prediction error of each bounding box component. The function $smooth_{L1}(\cdot)$ is defined as:

$$smooth_{L1}(r) = \begin{cases} 0.5r^2, & \text{if } |r| < 1, \\ |r| - 0.5, & \text{otherwise.} \end{cases} \tag{7}$$

In this formula, r denotes the residual error between the predicted value and the ground-truth value. When the error is small, i.e., $|r| < 1$, the loss uses the quadratic term $0.5r^2$, similar to L_2 loss. This makes the loss function smooth around zero and encourages precise localization. When the error is large, the loss becomes linear as $|r| - 0.5$, similar to L_1 loss. This prevents extremely large errors from dominating the training process. Therefore, Smooth L1 loss is suitable for bounding box regression because it provides stable optimization while remaining robust to large localization errors.

Generalized Intersection over Union Loss. Generalized Intersection over Union loss, abbreviated as GIoU loss, was proposed by Rezatofighi et al. in 2019 for bounding box regression. It extends the standard Intersection over Union (IoU) metric and addresses one important limitation of IoU. When two boxes do not overlap, their IoU is equal to zero, regardless of how far they are from each other. As a result, IoU cannot provide useful optimization information in non-overlapping cases. GIoU solves this problem by introducing an additional penalty term based on the smallest enclosing box that contains both the predicted box and the ground-truth box.

Let B be the predicted bounding box and $\hat{B}$ be the ground-truth bounding box. The standard IoU is computed as:

$$IoU(B, \hat{B}) = \frac{|B \cap \hat{B}|}{|B \cup \hat{B}|}, \tag{8}$$

where $|B \cap \hat{B}|$ is the area of overlap between the two boxes, and $|B \cup \hat{B}|$ is the area of their union. A higher IoU indicates a better overlap between the predicted box and the ground-truth box.

To compute GIoU, let C be the smallest enclosing box that covers both B and $\hat{B}$. The GIoU is defined as:

$$GIoU(B, \hat{B}) = IoU(B, \hat{B}) - \frac{|C - (B \cup \hat{B})|}{|C|}. \tag{9}$$

In this equation, the first term is the standard IoU, which measures the overlap between the predicted and ground-truth boxes. The second term is a penalty term. Here, $|C|$ is the area of the smallest enclosing box, and $|C - (B \cup \hat{B})|$ represents the area inside C that is not covered by either the predicted box or the ground-truth box. If the two boxes are close to each other, this penalty is small. If they are far apart, the enclosing box C becomes much larger, and the penalty increases. Therefore, GIoU can still provide meaningful supervision even when the predicted box and the ground-truth box do not overlap.

The corresponding GIoU loss is defined as:

$$\mathcal{L}_{GIoU}(B, \hat{B}) = 1 - GIoU(B, \hat{B}). \tag{10}$$

In TransVG, Smooth L1 loss and GIoU loss are combined to train the bounding box regression objective:

$$\mathcal{L} = \mathcal{L}_{smooth\text{-}L1}(b, \hat{b}) + \lambda \mathcal{L}_{GIoU}(b, \hat{b}), \tag{11}$$

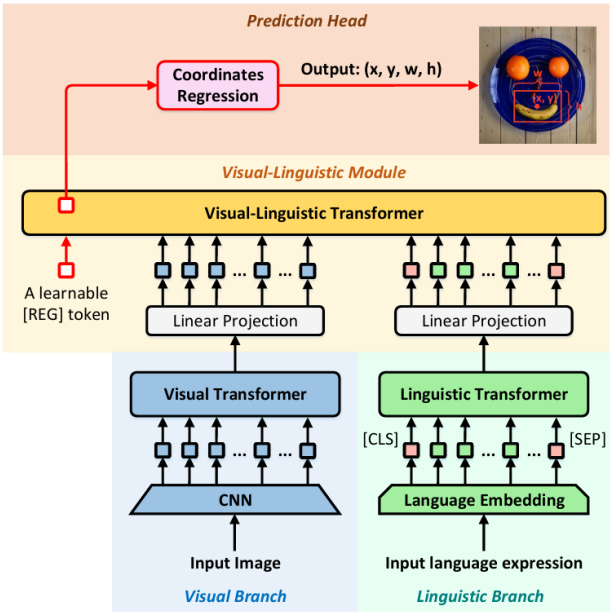


Fig. 2. An overview of our proposed TransVG framework. It consists of four main components: (1) a visual branch, (2) a linguistic branch, (3) a visual-linguistic fusion module, and (4) a prediction head to regress the box coordinates

where λ is a balancing coefficient used to control the contribution of the GIoU loss. Smooth L1 loss focuses on reducing coordinate-level prediction errors, while GIoU loss considers the geometric overlap and spatial relationship between the predicted and ground-truth boxes. By combining these two losses, the model can learn both accurate coordinate regression and better bounding box alignment.

C. Overall Architecture

As shown in Fig. 2, the reimplemented TransVG model consists of four main components: a visual branch, a linguistic branch, a visual-linguistic transformer, and a coordinate regression head. The visual branch extracts image features, while the linguistic branch encodes the input expression. The visual and linguistic features are then projected into a shared embedding space and fused by transformer encoder layers. Finally, a learnable regression token is used to predict the bounding box coordinates.

Visual Branch. The visual branch uses a convolutional backbone network, specifically ResNet, to extract a two-dimensional visual feature map from the input image. Continually, the visual transformer which is composed of a stack of 6 transformer encoder layers

Given an image $z_0 \in \mathbb{R}^{3\times H_0\times W_0}$, the backbone produces a 2D feature map $z \in \mathbb{R}^{C\times H\times W}$, where $C = 2048$ and the spatial size is usually reduced to $\frac{1}{32}$ of the original image size. A 1×1 convolution is then applied to reduce the channel dimension to $C_v = 256$. Since a transformer takes a sequence of vectors as input, the feature map is flattened into visual tokens $z_v \in \mathbb{R}^{C_v\times N_v}$, where $N_v = H \times W$. Positional encoding is added to preserve spatial information. Finally, the visual transformer outputs the enhanced visual embedding f_v.

Linguistic Branch. The linguistic branch includes a token embedding layer and a linguistic transformer. The architecture of linguistic transformer follows the design of the basic model of BERT series. Typically, there are 12 transformer encoder layers in the linguistic transformer. The output channel dimension of the linguistic transformer is $C_l = 768$

Given a language expression as the input of this branch, the authors first convert each word ID into a one-hot vector. Then, in the token embedding layer, the authors tokenize each one-hot vector into a language token by looking up the token table. The authors append a [CLS] token and a [SEP] token at the beginning and end positions of the tokenized language expression. After that, the authors take the language tokens as inputs of the linguistic transformer, and generate the advanced language embedding $f_l \in \mathbb{R}^{C_l\times N_t}$ where N_t is the number of language tokens.

Visual-Linguistic Fusion Module. This is the core component in TransVG model to fuse multi-model context, the architecture of visual-linguistic fusion module consists of two linear projection layers (one for each modality) and a visual-linguistic transformer (with a stack of 6 transformer encoder layers).

Given the visual tokens $f_v \in \mathbb{R}^{256\times N_v}$ and linguistic tokens $f_l \in \mathbb{R}^{768\times N_l}$, two linear projection layers are used to map them into the same channel dimension $C_p = 256$. The projected visual and linguistic embeddings are denoted as $p_v \in \mathbb{R}^{C_p\times N_v}$ and $p_l \in \mathbb{R}^{C_p\times N_l}$, respectively. A learnable $[REG]$ token $p_r \in \mathbb{R}^{C_p\times 1}$ is then appended to the joint sequence:

$$x_0 = [p_v^1, p_v^2, \ldots, p_v^{N_v}, p_l^1, p_l^2, \ldots, p_l^{N_l}, p_r].$$

The joint tokens are fed into the visual-linguistic transformer, where self-attention allows each visual and linguistic token to interact freely with others. Therefore, the model can perform both intra-modality and inter-modality reasoning without manually designed fusion rules. The output state of the $[REG]$ token contains the fused visual-linguistic representation and is used for box prediction.

Prediction Head and Training Objective. The prediction head takes the output representation of the $[REG]$ token from the visual-linguistic transformer as input. It uses a regression block implemented by an MLP with two ReLU-activated 256-dimensional hidden layers and a final linear layer. The final output is a 4-dimensional vector representing the bounding box coordinates of the referred object.

After having the prediction, the training objective of TransVG model is:

$$\mathcal{L} = \mathcal{L}_{smooth-L1}(b, \hat{b}) + \lambda \mathcal{L}_{GIoU}(b, \hat{b}) \tag{12}$$

where b is the predicted box, $\hat{b}$ is the ground-truth box, and λ controls the weight of the GIoU loss.

D. Limitations of TransVG

Although TransVG achieves strong performance and simplifies the visual grounding pipeline, it still has several limitations:

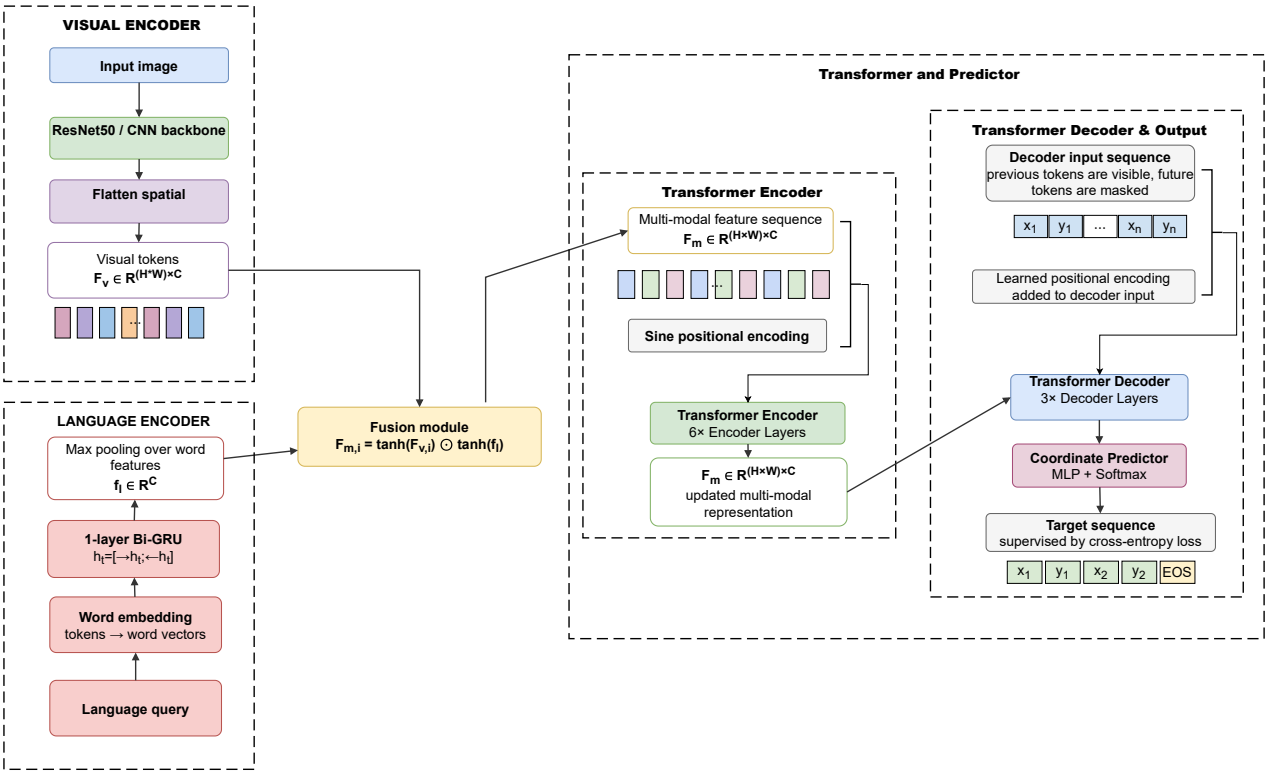


Fig. 3. Overview of the proposed network, It consists of four main components: (1) a visual branch, (2) a linguistic branch, (3) a fusion module, (4) a transformer and Predictor

- Its training objective is relatively complex because it combines Smooth L1 loss and GIoU loss for bounding box regression. This requires an additional balancing coefficient between the two losses and may make the optimization process less straightforward than using a single unified objective.
- TransVG mainly relies on transformer encoder layers, while the original Transformer architecture consists of both an encoder and a decoder. As a result, the model may not fully exploit the potential of decoder-based sequential modeling, which can be useful for generating structured outputs.
- TransVG contains a large number of parameters due to its use of heavy pre-trained components, including the DETR-based visual branch and the BERT-based linguistic branch. In particular, the visual transformer contains multiple encoder layers, and the linguistic branch follows the BERT architecture with 12 transformer encoder layers. For visual grounding, where the input language query is often a short referring expression, using such a large language model may be computationally inefficient.

V. Proposed Method: Decoder-Inspired Improvement

A. Premilinary

This section has the same function to the previous premilinary. In this section, we will introduce tools or knowledge used in the proposed method thus facilitating for readers to easily access to our architecture

Transformer Decoder. Before describing the proposed sequential coordinate prediction module, we briefly introduce the transformer decoder. Similar to the transformer encoder, the transformer decoder is also composed of a stack of attention-based layers. However, while the encoder only models the relationship among input tokens, the decoder is designed to generate an output sequence step by step by attending to both the previously generated tokens and the encoded input features.

Concretely, each transformer decoder layer contains three main sub-layers: a masked multi-head self-attention layer, a cross-attention layer, and a feed-forward network (FFN). The masked self-attention layer allows each output token to attend only to its previous tokens, preventing the model from accessing future information during training. This property is essential for autoregressive sequence generation. The cross-attention layer then uses the decoder features as queries and the encoder output features as keys and values, allowing the decoder to retrieve relevant information from the encoded visual-linguistic representation. Finally, the FFN further transforms the attended features with fully connected layers and non-linear activation functions.

Let y_n denote the input representation of the n-th transformer decoder layer, and let F_m denote the multi-modal feature representation produced by the encoder. The computation

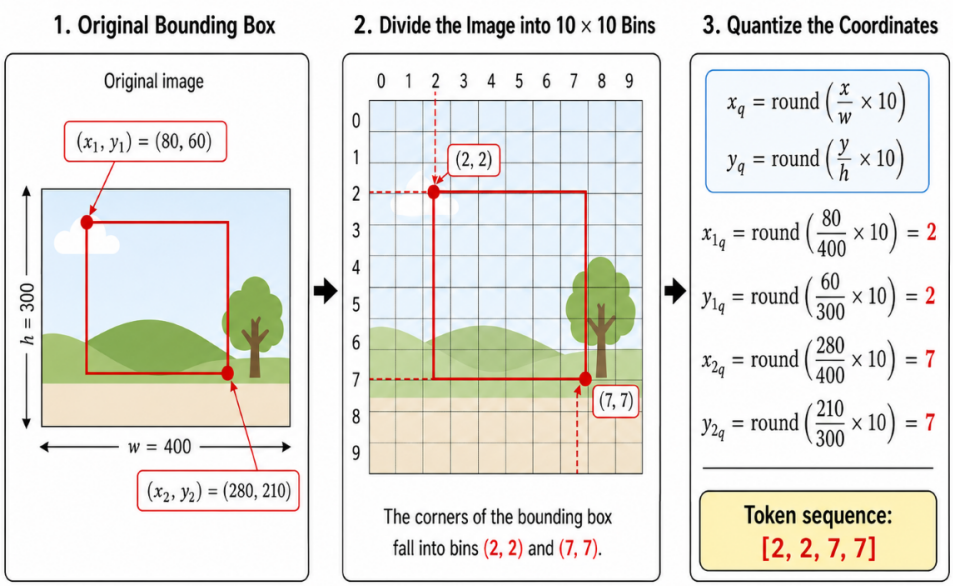


Fig. 4. This picture describes in detail of how to transform bounding box into a sequence of discrete coordinate tokens in order to help readers to easily understand the way of coordinates quantization

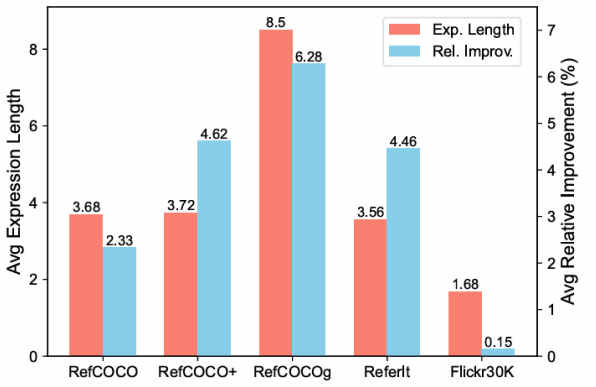


Fig. 5. The average expression length. These figures were referenced from SimVG article

process of a transformer decoder layer can be written as:

$$y'_n = \text{LN}(y_n + F_{\text{MSA}}(y_n)), \tag{13}$$

$$y''_n = \text{LN}(y'_n + F_{\text{CA}}(y'_n, F_m)), \tag{14}$$

$$y_{n+1} = \text{LN}(y''_n + F_{\text{FFN}}(y''_n)), \tag{15}$$

where $\text{LN}(\cdot)$ denotes layer normalization, $F * \text{MSA}(\cdot)$ represents the masked multi-head self-attention layer, $F * \text{CA}(\cdot)$ denotes the cross-attention layer, and $F_{\text{FFN}}(\cdot)$ represents the feed-forward network.

In the proposed method, the transformer decoder is used to generate bounding box coordinates as a sequence of discrete tokens. Given the multi-modal features from the encoder and the previously generated coordinate tokens, the decoder predicts the next coordinate token in an autoregressive manner. Therefore, instead of directly regressing continuous bounding box coordinates as in TransVG, the proposed decoder formulates visual grounding as a sequence prediction problem. This design enables the model to capture dependencies among coordinate tokens, such as the relationship between the top-left and bottom-right corners of the bounding box.

Bidirectional Gated Recurrent Unit. Recurrent Neural Networks (RNNs) are a family of neural networks designed to process sequential data, such as natural language sentences. Unlike feed-forward neural networks, an RNN maintains a hidden state that is updated step by step when reading a sequence. Therefore, the hidden state at each time step can store information from previous tokens. However, standard RNNs often suffer from the vanishing gradient problem when modeling long sequences, making it difficult to capture long-range dependencies.

To address this limitation, the Gated Recurrent Unit (GRU) was introduced as a gated variant of RNN. A GRU controls the information flow through two gates: an update gate and a reset gate. These gates allow the model to decide how much previous information should be kept and how much new information should be added at each time step. Compared with Long Short-Term Memory (LSTM), GRU has a simpler structure because it does not use a separate memory cell. As a result, GRU usually requires fewer parameters and is computationally more efficient, while still being effective for sequence modeling.

Given an input token representation x_t at time step t and the previous hidden state h_{t-1}, a GRU updates its hidden state as follows:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z), \tag{16}$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r), \tag{17}$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h), \tag{18}$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t. \tag{19}$$

In these equations, z_t is the update gate, r_t is the reset gate, and $\tilde{h}_t$ is the candidate hidden state. The function $\sigma(\cdot)$ denotes

the sigmoid activation function, which outputs values between 0 and 1. The operator $\odot$ denotes element-wise multiplication. The matrices W_z, W_r, and W_h are used to transform the current input x_t, while U_z, U_r, and U_h are used to transform the previous hidden state h_{t-1}. The terms b_z, b_r, and b_h are bias vectors.

The update gate z_t determines how much information from the previous hidden state should be preserved and how much new information should be written into the current hidden state. If the value of z_t is large, the model gives more importance to the candidate hidden state $\tilde{h}_t$. If the value of z_t is small, the model keeps more information from the previous hidden state h_{t-1}. The reset gate r_t controls how much past information should be used when computing the candidate hidden state. This mechanism helps the GRU selectively forget irrelevant previous information and focus on the current context.

Although a standard GRU can capture the left-to-right context of a sentence, it only processes the sequence in one direction. For language understanding tasks, the meaning of a word often depends not only on the words before it but also on the words after it. Therefore, a Bidirectional GRU (Bi-GRU) is used to capture contextual information from both directions. A Bi-GRU consists of two GRU layers: a forward GRU and a backward GRU. The forward GRU reads the sentence from left to right, while the backward GRU reads it from right to left.

For a sequence of word embeddings $\{x_t\}_{t=1}^{T}$, the forward and backward hidden states are computed as:

$$\overrightarrow{h}_t = \mathrm{GRU}_f(x_t, \overrightarrow{h}_{t-1}), \tag{20}$$

$$\overleftarrow{h}_t = \mathrm{GRU}_b(x_t, \overleftarrow{h}_{t+1}). \tag{21}$$

The final word representation at time step t is obtained by concatenating the hidden states from both directions:

$$h_t = [\overrightarrow{h}_t; \overleftarrow{h}_t]. \tag{22}$$

Here, $\overrightarrow{h}_t$ contains information from the previous tokens, while $\overleftarrow{h}_t$ contains information from the future tokens. By combining these two representations, Bi-GRU can produce a more complete contextual representation for each word in the referring expression. For example, in a visual grounding query, the representation of an object word may be influenced by its attributes, spatial relations, and surrounding words. This is important because the model needs to understand not only the target object but also the descriptive information that distinguishes it from other objects in the image.

In the proposed architecture, Bi-GRU is adopted as the language encoder to replace the heavy BERT-based linguistic branch. This choice helps reduce computational cost and the number of parameters. Since visual grounding queries are usually short referring expressions, a lightweight recurrent encoder can be sufficient to capture the necessary linguistic context. The output hidden states of the Bi-GRU are used as word-level features and are later fused with visual features for multi-modal reasoning. Therefore, Bi-GRU provides a practical balance between language representation ability and computational efficiency.

B. The Idea of Improvement

As shown in Fig 5, expressions of five dataset is relatively short, using such a large pre-trained model like BERT is waste of computing resource. To reduce the computational cost and the number of model parameters, we replace the heavy pre-trained language encoder used in TransVG with a more lightweight recurrent language encoder. Specifically, instead of using BERT to encode the referring expression, we adopt a traditional RNN-based architecture, namely a bidirectional GRU (Bi-GRU), to extract contextual word representations. This design is more suitable for visual grounding, where the input text is usually a short referring expression rather than a long document.

Furthermore, we reformulate visual grounding from a direct coordinate regression problem into a sequence generation problem. Instead of predicting the bounding box coordinates with a regression head, the target bounding box is represented as a sequence of discrete coordinate tokens. The transformer decoder is then used to generate these coordinate tokens in an auto-regressive manner, conditioned on the fused visual-linguistic features. In this way, the model can better exploit the encoder-decoder structure of the original Transformer architecture.

This sequence-based formulation also simplifies the training objective. While TransVG relies on a combination of Smooth L1 loss and GIoU loss for bounding box regression, the proposed method predicts discrete coordinate tokens and can therefore be optimized using a standard cross-entropy loss. As a result, the model avoids the need for multiple hand-crafted regression losses and additional loss-balancing coefficients, leading to a simpler and more unified optimization process.

C. Coordinate Quantization

Inspired by Pix2Seq, as shown in Fig 4 A key component is to serialize and quantize the bounding box into a sequence of discrete coordinate tokens, which enables grounding task to be addressed in network using transformer decoder. Let M be the number of quantization bins. Each continuous coordinate is normalized by the image width or height and mapped to an integer token:

$$x_i = \mathrm{round}\left(\frac{\tilde{x}_i}{W} \times M\right), \quad y_i = \mathrm{round}\left(\frac{\tilde{y}_i}{H} \times M\right), \tag{23}$$

where $(\tilde{x}_i, \tilde{y}_i)$ are the original floating-point coordinates, and W and H are the image width and height.

For bounding box grounding, the target sequence is defined as:

$$T = [x_1, y_1, x_2, y_2, EOS]. \tag{24}$$

D. Architecture Overview

As shown in Fig. 3, The proposed architecture consists of four main components: a language encoder, a visual encoder, a multi-modal fusion module, and a transformer-based coordinate decoder.

Linguistic Branch. For the language branch, instead of using a large pre-trained language model such as BERT, a lightweight one-layer bidirectional GRU is adopted to encode

TABLE I
MAIN QUANTITATIVE RESULTS

Method	Backbone	Precision@0.5			Notes
		Val	TestA	TestB	
Original TransVG (90 epochs)	ResNet-50	80.32	82.67	78.12	Reported/official result from the original paper.
Reimplemented TransVG (30 epochs)	ResNet-50	46.58	51.32	39.23	Our reproduced baseline under limited hardware resources.
Proposed Method (30 epochs)	ResNet-50	74.65	79.32	67.67	Our proposed method.

the referring expression. The hidden states from the forward and backward GRU are concatenated at each time step to form contextual word representations.

Visual Branch. For the visual branch, the image is processed by a visual encoder to extract visual features. The visual features are down-sampled and flattened into a sequence $F_v \in \mathbb{R}^{(H\times W)\times C}$, where H and W are smaller than the original image size. These visual features are then used as the input to the fusion module.

Fusion. To align the visual and linguistic modalities, a simple fusion strategy is applied. Given the visual features F_v and word features $\{h_t\}_{t=1}^{T}$ The word features are first aggregated into a global language feature $f_l \in \mathbb{R}^C$ by max pooling. Then, the visual feature at each spatial position is fused with the language feature using the Hadamard product:

$$F_{m,i} = \sigma(F_{v,i}) \odot \sigma(f_l),$$

where σ denotes the tanh activation function and $F_m \in \mathbb{R}^{(H*W)\times C}$ represents the fused multi-modal features. This design avoids directly concatenating all visual and word tokens, which helps reduce computational complexity.

Transformer and Predictor. After fusion, the multi-modal features are passed into a transformer encoder to update their representations. Then, a transformer decoder predicts the bounding box as a sequence of discrete coordinate tokens in an auto-regressive manner. Specifically, the decoder generates the coordinates step by step, conditioned on the fused multi-modal features and the previously predicted coordinate tokens. The hidden dimension of transformer is set to 256, the expansion rate in feed forward network (FFN) is 4, and the number of encoder and decoder layers are 6 and 3, respectively. Sine positional encodings are added to the multi-modal features, while learned positional encodings are used for the decoder input sequence. Finally, an MLP followed by a softmax layer is used to predict each coordinate token. During inference, the predicted coordinate tokens are mapped back to the original image scale and assembled into the final bounding box.

E. Training Objective

The proposed model is optimized with cross-entropy loss over the coordinate vocabulary:

$$\mathcal{L}_{Seq} = -\sum_{i=1}^{K} \log P(T_i \mid F_m, s_{1:i-1}), \tag{25}$$

where F_m denotes the multi-modal feature representation, T_i is the target token at step i, and K is the target sequence length ($K = 4$ denotes the number of coordinate tokens, excluding the EOS token for bounding box). Note that the input sequence $S_{1:i-1}$ only contains the preceding coordinate tokens when predicting the $i-th$ one.

VI. DATASET

This section introduces five benchmark datasets commonly used for visual grounding. Five datasets include: ReferItGame, Flickr30K Entities, RefCOCO/RefCOCO+/RefCOCOg. However, due to limitations of hardware resources, we are only able to conduct experiments on RefCOCO dataset.

A. Dataset Description

ReferItGame. ReferItGame includes 20,000 images collected from the SAIAPR-12 dataset, and each images has one or a few regions with corresponding referring expressions

Flickr30K Entities. Flickr30K Entities augments the original Flickr30K with short region phrase correspondence annotations. It contains 31,783 images with 427K referred entities.

RefCOCO/RefCOCO+/RefCOCOg. RefCOCO includes 19,994 images with 50,000 referred objects. Each object has more than one referring expression, and there are 142,210 referring expressions in this dataset. The samples in RefCOCOareofficially split into a train set with 120,624 expressions, a validation set with 10,834 expressions, a testA set with 5,657 expressions and a testB set with 5,095 expressions. Similarly, RefCOCO+ contains 19,992 images with 49,856 referred objects and 141,564 referring expressions. It is also officially split into a train set with 120,191 expressions, a validation set with 10,758 expressions, a testA set with 5,726 expressions and a testB set with 4,889 expressions. RefCOCOg has 25,799 images with 49,856 referred objects and expressions. There are two commonly used split protocols for this dataset. One is RefCOCOg-google, and the other is RefCOCOg umd.

VII. EXPERIMENTAL SETUP

A. Implementation Details

1) Image and Text Preprocessing: Images are resized to 640×640 while preserving aspect ratio. The bounding box coordinates are scaled accordingly. Language expressions are tokenized and padded or truncated to a maximum length of 15 tokens. Padding masks are used to prevent the model from attending to invalid image or text tokens.

TABLE II
TRAINING CONFIGURATION

Setting	Value
GPU	T4
Backbone	ResNet50
Image size	640×640
Max text length	15
Batch size	16
Epochs	30
Optimizer	AdamW
Learning rate	Vary from each module

2) Training Detail: Due to the limitation of available computational resources, we were only able to reproduce the training of TransVG for 30 epochs, instead of 90 epochs as reported in the original setting. For a fair comparison, the same number of training epochs was also applied to our proposed method. For TransVG, we followed the learning rate configuration of the original paper, which is 10^{-4} for initial learning rate of the Visual Linguistic module and prediction head and 10^{-5} for the visual branch and linguistic branch. However, since our batch size was smaller than the original setting, the learning rates were reduced accordingly to maintain stable training. For the proposed method, we used a learning rate of 3.125×10^{-5}. In addition, ResNet50 was adopted as the visual backbone for both methods to ensure a fair comparison under the same feature extraction setting.We summarize the main training configuration in Table II

B. Compared Methods

The following methods are compared in the experiments:

- **Original TransVG**: results reported in the original paper or official implementation.
- **Reimplemented TransVG**: our reproduced baseline under the same dataset setting.
- **Proposed Method**: the TransVG encoder combined with a Decoder-inspired sequential coordinate decoder.

C. Evaluation Metrics

The primary evaluation metric is Precision@0.5. The prediction is deemed correct if its intersection over union (IoU) with ground-truth box is larger than 0.5. We also report mean IoU and qualitative prediction examples to analyze model behavior more comprehensively. Let we show you the math formula of IoU once again, B be the predicted bounding box and $\hat{B}$ be the ground- truth bounding box. :

$$IoU(B, \hat{B}) = \frac{|B \cap \hat{B}|}{|B \cup \hat{B}|}, \tag{26}$$

where $|B \cap \hat{B}|$ is the area of overlap between the two boxes, and $|B \cup \hat{B}|$ is the area of their union.

VIII. RESULTS AND DISCUSSION

A. Quantitative Results

Table I presents the quantitative comparison between the original TransVG result, our reproduced TransVG model, and the proposed method on the RefCOCO dataset. All methods use ResNet-50 as the backbone to ensure a fair comparison in terms of visual feature extraction. The original TransVG, trained for 90 epochs as reported in the original paper, achieves the best performance with 80.32%, 82.67%, and 78.12% Precision@0.5 on the validation, TestA, and TestB sets, respectively.

Due to hardware limitations, our reproduced TransVG model was trained for only 30 epochs. As a result, its performance drops significantly to 46.58% on the validation set, 51.32% on TestA, and 39.23% on TestB. Under the same 30-epoch training setting, the proposed method achieves 74.65%, 79.32%, and 67.67% on the validation, TestA, and TestB sets, respectively. Compared with the reproduced TransVG model, the proposed method improves the performance by 28.07%, 28.00%, and 28.44% on the three splits.

These results show that the proposed method achieves substantially better performance than the reproduced TransVG baseline under the same limited training condition. Although it is still lower than the fully trained original TransVG, the proposed method obtains competitive results with fewer training epochs, demonstrating the effectiveness of reformulating visual grounding as a sequence generation problem and using a lighter language encoder.

B. Ablation Study

In this section, we will present ablation study of TransVG again as well as conduct ablative experiments about our proposed method on the validation set of the RefCOCO dataset.

1) TransVG: The authors conduct ablative experiments to verify the effectiveness of each component in the TransVG framework, all of the compared models are trained for 90 epochs.

Design of the [REG] Token. The authors study the design of the [REG] token on RefCOCO dataset, and report the results in Table IV. There are several choices to construct the initial state of the [REG] token.We detail these designs and analysis them as follows:

- Average pooled visual tokens. The authors perform average pooling over the viual tokens and exploit the average pooled embedding as the initial sate of [REG] token.
- Max pooled visual tokens. We take the max-pooled visual token embedding as the initial [REG] token.
- Average pooled linguistic tokens. Similarity to the first choice, but using the linguistic tokens.
- Average pooled linguistic tokens. Similarity to the second choice, but using the linguistic tokens
- Sharing with [CLS] token. We use the [CLS] token off linguistic embedding to pl the [REG] token. Concretely, the [CLS] token out of the V-L module is fed into the prediction head.
- Learnable embedding. Randomly initializing the [REG] token embedding at the beginning off the training stage. And the parameters of this embedding are optimized with the whole model

The authors exploit a learnable embedding achieves 80.32% top-1 accuracy on the validation set of RefCOCO, which is the best performance among all the designs

pp

TABLE III
ABLATION STUDY ON VISUAL AND LINGUISTIC TRANSFORMERS

Visual Branch		Linguistic Branch		Accuracy (%)	Runtime (ms)
w/o Tr.	w/ Tr.	w/o Tr.	w/ Tr.		
✓		✓		64.24	33.67
✓			✓	$66.78_{\uparrow 3.54}$	47.57
	✓	✓		$68.48_{\uparrow 4.24}$	40.14
	✓		✓	$\mathbf{69.76}_{\uparrow 5.52}$	61.77

TABLE IV
ABLATION STUDY ON THE INITIAL STATE OF THE [REG] TOKEN

Initial State of [REG] Token	RefCOCO@val
Average pooled visual tokens	79.12
Max pooled visual tokens	78.37
Average pooled linguistic tokens	78.51
Max pooled linguistic tokens	78.74
Sharing with [CLS] token	77.90
Learnable embedding*	**80.32**

Transformers in Visual and Linguistic Branches. The authors study the role of transformers in the visual branch and the linguistic branch. Table III summarizes the results of several models with or without the visual transformer and the linguistic transformer. The baseline model without both visual transformer and linguistic transformer reports an accuracy of 64.24%. When the authors only attach either the visual transformer or the linguistic transformer, an improvement of 68.48% and 66.78% are achieved, respectively. With the complete architecture, the performance is further boosted to 69.76% on the ReferIt test set. This result demonstrates the essential of transformers in the visual branch and linguistic branch to capture intra-modality global context before performing multi-modal fusion

2) Our Proposed Method: Table V and Table VI presents ablation experiments for the proposed method. To evaluate the effectiveness of different design choices in our proposed method, we conduct a series of ablation studies focusing on two key components of the model: the construction of language features (from the bidirectional GRU language encoder) and the token-specific loss weights (applied to the output sequence).

All ablation models are trained 5 epochs from scratch under identical hyperparameter settings (e.g., learning rate, batch size, optimizer) to ensure fair comparison

Construction of language feature. The language encoder plays a critical role in visual grounding as it extracts semantic representations from the textual query to guide the visual attention mechanism. In our model, the query is first processed by a Bidirectional Gated Recurrent Unit (bi-GRU), which yields a sequence of hidden states $H = \{h_1, h_2, h_3, ..., h_L\}$, where L is the sequence length (padded to a maximum length of 15 tokens) and $h_t \in \mathbb{R}^{2\times d_{gru}}$ represents the concatenated forward and backward hidden states of the i-th token. To obtain a single, global language feature vector$y \in \mathbb{R}^{2\times d_{gru}}$ that represents the entire query, we explore three different aggregation (pooling) methods:

- Max Pooling: In this approach, we apply a element-wise operation across the sequence dimension, excluding padding tokens. The padding tokens are masked by replacing their feature with negative infinity before pooling to ensure they do not affect the output $-\infty$
- Average Pooling: We average the hidden states across the sequence dimension, considering only the valid (non-padded) tokens. $y = \frac{1}{N_{valid}} \sum_{i\in valid} h_t$
- Last Hidden State: We extract the final state of the sequence by concatenating the last hidden state of the forward GRU and the first hidden state of the backward GRU (which corresponds to the final output of the backward pass). $y = [h_L^{->}; h_L^{<-}]$

As shown in Table V, Among the three compared methods, max pooling achieves the best performance, with 22.81% Precision@0.5 and 32.20 mIoU. Compared with mean pooling, max pooling improves Precision@0.5 from 21.45% to 22.81% and mIoU from 31.48 to 32.20. This indicates that max pooling is more effective in selecting discriminative semantic information from the word-level hidden states. In contrast, using the final hidden state produces the lowest performance, with only 19.72% Precision@0.5 and 29.77 mIoU. This result suggests that relying only on the final hidden state may lose important information from earlier tokens, especially when the referring expression contains multiple attributes or spatial relationships.

Token weight in loss function. We reformulates visual grounding as a sequence generation task. The model predicts a 5-token target sequence: $T = [x_1, y_1, x_2, y_2, EOS]$ where (x_1, x_2) represents the top-left corner, (x_2, y_2) represents the bottom-right corner of the bounding box, and EOS is the sequence termination token. In an autoregressive decoder, early predictions directly condition subsequent predictions. If the model predicts an incorrect start coordinate, this error propagates and inevitably degrades the precision of the remaining coordinates (y_1, x_2, y_2). To address this exposure bias and encourage the model to focus on critical anchor points, we apply token-specific weights to the cross-entropy loss: $L_{seq} = \frac{1}{5}\sum_{i=1}^{5} w_t \cdot L_{CE}(p_t, g_t)$ where w_t is the weight of the $t-th$ token in sequence, p_t is the predicted probability distribution, and g_t is the ground-truth token. We investigate four difference token weight schemas:

- Uniform weights $w = [1.0, 1.0, 1.0, 1.0, 1.0]$. Every token in the sequence contributes equally to the total loss. This represents the standard cross-entropy loss setting.
- $w = [1.5, 1.0, 1.0, 1.0, 1.0]$. A higher weight (1.5) is assigned to the first token (x_1). This penalizes early errors more heavily, establishing a strong anchor point for the bounding box generation.
- $w = [1.5, 1.5, 1.0, 1.0, 1.0]$. This scheme extends the penalty to the entire start coordinate pair (x_1, y_1), forcing the model to locate the top-left bounding box coordinate with high precision.
- Decaying weights $w = [2.0, 1.5, 1.0, 1.0, 0.5]$. A step-wise decay across the sequence. The starting coordinates are prioritized, the ending coordinates receive standard weight, and the termination token (EOS) is penalized least.

TABLE V
ABLATION STUDY OF THE CONSTRUCTION LANGUAGE FEATURE

Language feature	Precision@0.5	mIoU
mean pooling	21.45	31.48
max pooling	**22.81**	**32.22**
final hidden state	19.72	29.77

TABLE VI
ABLATION STUDY ON TOKEN WEIGHT

Token Weight					Precision@0.5	mIoU
1st	2nd	3rd	4th	5th		
1	1	1	1	1	22.40	31.85
1.5	1	1	1	1	22.12	31.63
1.5	1.5	1	1	1	22.67	32.11
2	1.5	1	1	0.5	**23.32**	**32.47**

As shown in Table VI, the uniform weighting setting, where all coordinate tokens have the same weight, achieves 22.40% Precision@0.5 and 31.85 mIoU. Simply increasing the first token weight to 1.5 does not improve the result, slightly decreasing the performance to 22.12% Precision@0.5 and 31.63 mIoU. However, assigning larger weights to the first two coordinate tokens improves the performance to 22.67% Precision@0.5 and 32.11 mIoU. The best result is obtained when the token weights are set to $(2, 1.5, 1, 1, 0.5)$, reaching 23.32% Precision@0.5 and 32.47 mIoU. This setting improves the uniform baseline by 0.92 percentage points in Precision@0.5 and 0.62 in mIoU

C. Qualitative Results

Proposed Method. Figure 6 presents qualitative examples of the proposed method on three different RefCOCO splits, including validation, testA, and testB. Overall, the model is able to localize the referred objects reasonably well across different types of expressions and visual scenes. In the validation split, the model successfully grounds diverse object categories such as a motorcycle, sheep, giraffe, and truck. These examples show that the model can associate object-level visual patterns with the corresponding language queries. In the testA split, which mainly contains person-related expressions, the predictions also show good alignment with the target regions. For example, the model can distinguish the referred person based on appearance, position, or contextual information in the query.

The testB examples are more challenging because they usually involve non-person objects, small targets, or objects that are visually similar to nearby regions. Nevertheless, the model still produces reasonable predictions in several cases, such as the bus, the object on the plate, and the orange on the right. Some imperfect predictions can also be observed. In these cases, the predicted box may slightly shift from the ground-truth region or cover only part of the target object. This usually happens when the target object is small, partially occluded, visually ambiguous, or surrounded by similar objects. These qualitative results indicate that the proposed method has learned meaningful cross-modal correspondence between language expressions and visual regions, but it still faces difficulties in fine-grained localization and ambiguous referring expressions.

TransVG. Figure 7 presents qualitative results of the reimplemented TransVG model on three RefCOCO splits, including validation, testA, and testB. Overall, the model is able to produce reasonable localization results in several cases, especially when the referred object has a clear visual appearance and occupies a relatively large region in the image. In the validation split, the model successfully localizes objects such as the train, the batter, and the person wearing a blue jacket with relatively high IoU scores. These examples indicate that the reimplemented TransVG can learn meaningful visual-linguistic correspondence and can associate language expressions with the corresponding visual regions.

However, the results also reveal several limitations of the reimplemented baseline. In some examples, the predicted bounding box is shifted from the ground-truth region or only covers part of the referred object. This can be observed in cases where the target is small, partially occluded, or difficult to distinguish from nearby objects. In the testA split, which mainly contains person-related expressions, the model can sometimes locate the correct person category but still fails to precisely cover the target instance. For example, the predicted boxes may include only part of the person or may be biased toward a nearby person with similar appearance. This suggests that distinguishing among multiple similar human instances remains challenging.

The testB split appears more difficult, since it often contains non-person objects, small objects, or objects with ambiguous visual boundaries. Several examples in the bottom row show relatively low IoU scores, such as the zebra, the cat on the shelf, and the yellow object with blue. These failure cases suggest that direct coordinate regression in TransVG may struggle with fine-grained localization when the target object is small, occluded, or described by subtle attributes. Although the model can roughly attend to the relevant image region, it does not always predict a bounding box that tightly matches the ground truth. This observation motivates the need for further improvement, such as the Decoder-inspired sequential coordinate prediction method, which may better model dependencies between bounding box coordinates and reduce the difficulty of direct continuous regression.

IX. LIMITATIONS

The project has several limitations:

- The reimplementation may not exactly reproduce the original training setting due to hardware constraints, dataset differences, or unavailable pre-trained weights.
- the proposed method introduces autoregressive prediction, which may increase inference time compared with direct regression.
- Coordinate quantization may introduce discretization errors, especially when the number of bins is small
- The experiments are limited to RefCOCO dataset and should be extended to standard benchmarks such as RefCOCO+, RefCOCOg, ReferItGame, and Flickr30K Entities.

X. Conclusion

This report presented a project on reimplementing TransVG and improving it with a Decoder-inspired sequential coordinate prediction method. The reimplementation helped clarify the role of transformer-based visual-linguistic fusion in visual grounding. The proposed improvement reformulated bounding box localization as coordinate token generation and replaced the regression head with a transformer decoder trained by cross-entropy loss. Experimental results show that the proposed method outperforms our reimplemented TransVG baseline under the same 30-epoch training setting on RefCOCO. Future work includes improving the decoder design, exploring larger datasets, applying stronger data augmentation, and extending the method to referring expression segmentation.

References

[1] J. Deng, Z. Yang, T. Chen, W. Zhou, and H. Li, "TransVG: End-to-End Visual Grounding with Transformers," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021.

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is All You Need," in *Advances in Neural Information Processing Systems*, 2017.

[3] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-End Object Detection with Transformers," in *Proceedings of the European Conference on Computer Vision*, 2020.

[4] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," arXiv preprint arXiv:1810.04805, 2018.

[5] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016.

[6] H. Rezatofighi, N. Tsoi, J. Gwak, A. Sadeghian, I. Reid, and S. Savarese, "Generalized Intersection over Union: A Metric and a Loss for Bounding Box Regression," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019.

[7] L. Yu, P. Poirson, S. Yang, A. C. Berg, and T. L. Berg, "Modeling Context in Referring Expressions," in *Proceedings of the European Conference on Computer Vision*, 2016.

[8] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling," arXiv preprint arXiv:1412.3555, 2014.

[9] M. Schuster and K. K. Paliwal, "Bidirectional Recurrent Neural Networks," *IEEE Transactions on Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997.

[10] L. Yu, Z. Lin, X. Shen, J. Yang, X. Lu, M. Bansal, and T. L. Berg, "MAttNet: Modular Attention Network for Referring Expression Comprehension," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018.

[11] Z. Yang, B. Gong, L. Wang, W. Huang, D. Yu, and J. Luo, "A Fast and Accurate One-Stage Approach to Visual Grounding," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019.

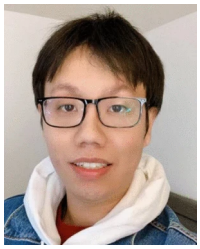
[12] J. Mao, J. Huang, A. Toshev, O. Camburu, A. L. Yuille, and K. Murphy, "Generation and Comprehension of Unambiguous Object Descriptions," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016.

[13] B. A. Plummer, L. Wang, C. M. Cervantes, J. C. Caicedo, J. Hockenmaier, and S. Lazebnik, "Flickr30K Entities: Collecting Region-to-Phrase Correspondences for Richer Image-to-Sentence Models," *International Journal of Computer Vision*, vol. 123, no. 1, pp. 74–93, 2017.

[14] S. Kazemzadeh, V. Ordonez, M. Matten, and T. Berg, "ReferItGame: Referring to Objects in Photographs of Natural Scenes," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, 2014.

Jiajun Deng Jiajun Deng is a Chinese researcher in the field of artificial intelligence. He is currently a Specially Appointed Professor at the University of Science and Technology of China (USTC). From 2012 to 2016, Jiajun Deng completed his Bachelor of Engineering degree in Electrical Engineering and Computer Science at the University of Science and Technology of China (USTC). He then continued his doctoral studies at USTC from 2016 to 2021, where he obtained his Ph.D. degree under the supervision of Professor Wengang Zhou and Professor Houqiang Li. After completing his Ph.D., Jiajun Deng worked as a Postdoctoral Researcher at the University of Adelaide, Australia, from 2021 to 2023. From 2023 to 2024, he continued his postdoctoral research at the National University of Singapore (NUS). His research mainly focuses on computer vision, vision-language understanding, and visual grounding, with particular attention to Transformer-based architectures for multimodal reasoning.

Zhengyuan Yang Zhengyuan Yang is currently a Member of Technical Staff at Microsoft AI Superintelligence. He earned his Ph.D. degree in Computer Science from the University of Rochester under the supervision of Prof. Jiebo Luo, and received his bachelor's degree from the University of Science and Technology of China (USTC). Dr. Yang is the recipient of several prestigious accolades, including the ACM SIGMM Award for Outstanding Ph.D. Thesis, the Twitch Research Fellowship, and the ICPR 2018 Best Industry Related Paper Award. His core research centers on multimodal foundation models, with a particular focus on post-training them to solve long-horizon tasks. Additionally, his work spans advanced topics in multimodal understanding and generation.



Tianlang Chen Tianlang Chen is currently an Applied Scientist at Amazon. He earned his Ph.D. degree from the Department of Computer Science at the University of Rochester, under the supervision of Prof. Jiebo Luo, and obtained his bachelor's degree in Electronic and Information Engineering from the University of Science and Technology of China (USTC), supervised by Prof. Xinmei Tian. Throughout his doctoral career, Dr. Chen gained extensive industry experience as a research intern at Google Research, Adobe Research, Bytedance AI Lab U.S., and Tencent Youtu Lab. His research interests lie at the intersection of Computer Vision, Natural Language Processing, and Deep Learning, with a specific focus on vision-language modeling.

Wengang Zhou Wengang Zhou is currently a Professor in the Department of Electronic Engineering and Information Science (EEIS) at the University of Science and Technology of China (USTC). He received his B.Sc. degree in Electronic Information Engineering from Wuhan University in 2006, and earned his Ph.D. degree in Signal and Information Processing from USTC in 2011, under the supervision of Prof. Houqiang Li. From 2011 to 2013, he worked as a Post-Doctoral Researcher in the Computer Science Department at the University of Texas at San Antonio, advised by Prof. Qi Tian. He returned to USTC as an Associate Professor in 2013 and was promoted to Professor in 2018. Dr. Zhou's primary research interests focus on Computer Vision and Embodied AI. He also serves as an Associate Editor for IEEE Transactions on Multimedia.

Houqiang Li Houqiang Li is a Professor and Ph.D. Supervisor in the Department of Electronic Engineering and Information Science at the University of Science and Technology of China (USTC). He serves as the Director of the MOE-Microsoft Key Laboratory of Multimedia Computing and Communication. Dr. Li is an IEEE Fellow, a recipient of the National Science Fund for Outstanding Young Scholars (2013), and a Changjiang Distinguished Professor (2017). He received his B.S., M.S., and Ph.D. degrees from USTC in 1992, 1997, and 2000, respectively, and completed his post-doctoral research there in 2002. His major research areas include computer vision, pattern recognition, artificial intelligence, video coding, and multimedia retrieval. Professionally, Dr. Li has held significant academic positions, including serving as General Co-Chair for IEEE ICME 2021, Program Co-Chair for VCIP 2010, and Associate Editor for top-tier journals such as IEEE Transactions on Multimedia and IEEE TCSVT. He is also an active member of international standardization bodies including MPEG, JCTVC, and JVT.

Khoi Le Minh Khoi Le Minh is an undergraduate student at the Faculty of Information Science and Engineering, University of Information Technology His research interests include computer vision, natural language processing, deep learning, and vision-language understanding.

Bao Bui Quoc Bao Bui Quoc is an undergraduate student at the Faculty of Information Science and Engineering, University of Information Technology His research interests include computer vision, natural language processing, deep learning, and vision-language understanding.

Dat Huynh Phat Dat Huynh Phat is an undergraduate student at the Faculty of Information Science and Engineering, University of Information Technology His research interests include computer vision, natural language processing, deep learning, and vision-language understanding.

Fig. 6. Qualitative results of the proposed method on three RefCOCO splits. The examples are arranged from top to bottom as validation, testA, and testB, respectively. Blue boxes denote predicted bounding boxes, while red boxes denote ground-truth boxes. The IoU score of each prediction is shown above the corresponding image.

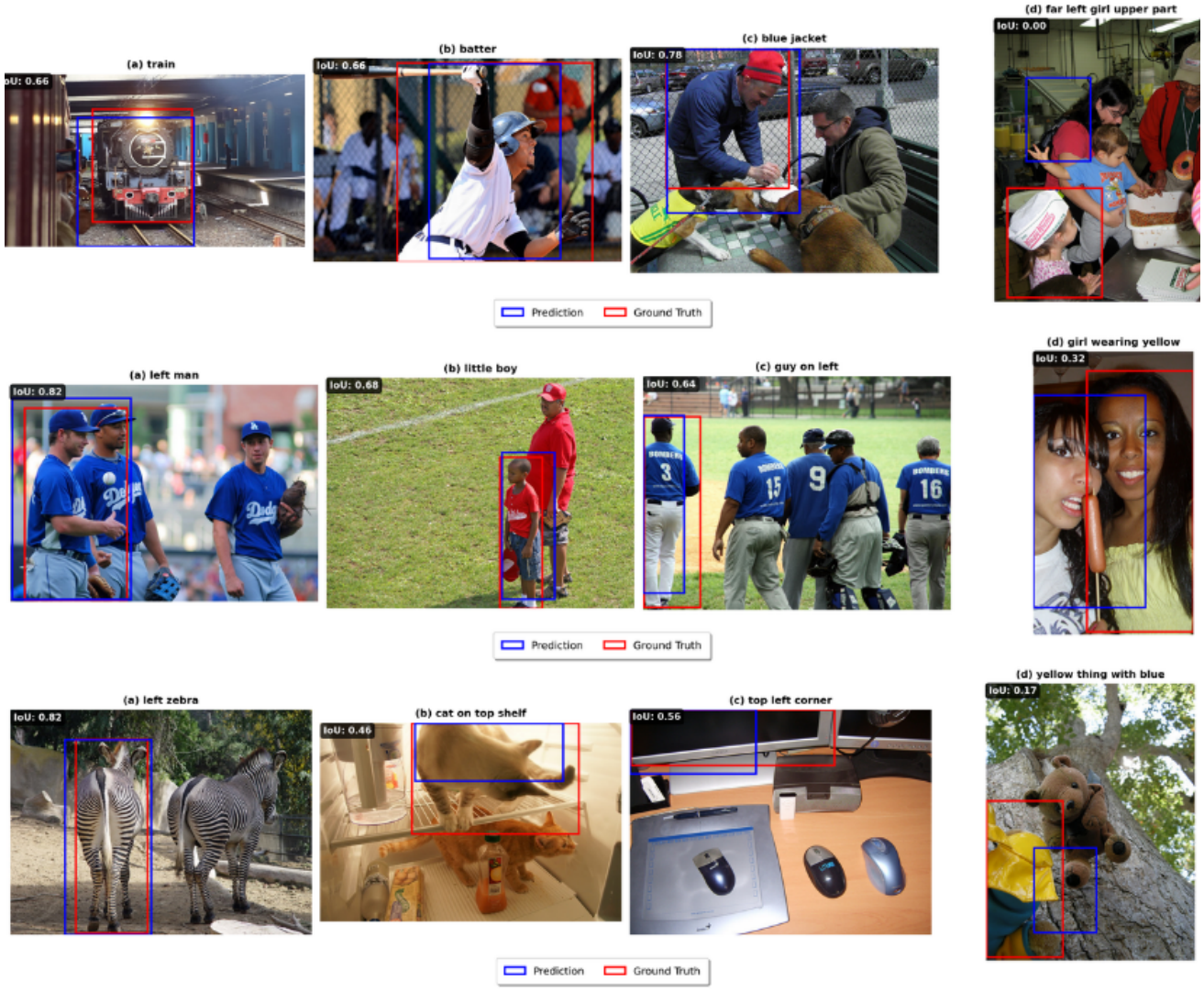


Fig. 7. Qualitative results of the reimplemented TransVG model on three RefCOCO splits. The examples are arranged from top to bottom as validation, testA, and testB, respectively. Blue boxes denote predicted bounding boxes, while red boxes denote ground-truth boxes. The IoU score of each prediction is shown above the corresponding image.